

Lab 4 Report

Ryan Wilson

Due: 02/17/2026 @ 11:00 AM

1 Exercise 1:

Problem: The goal for this exercise is to write 3 separate programs that help the user with I/O usage and error checking. The first program reads a file and prints the first 3 lines of it. The second program reads a file and prints the last 3 lines of it. The third program creates a new file if it doesn't already exist and writes integers 1 to 100 to it, with one number per line.

Design:

Pseudo code for head.c:

```
Prompt user for filename
Begins opening and reading file
If fopen() returns NULL, prints error and terminates
Reads first 3 lines using fgets()
Prints each of the first 3 lines after reading it
Closes and exits file
```

Pseudo code for tail.c:

```
Prompt user for filename
Begins opening and reading file
If fopen() returns NULL, prints error and terminates
Reads whole file line-by-line into array
Tracks number of lines in file
Prints each of the last 3 lines of the file
Closes and exits file
```

Pseudo code for out.c:

```
Prompt user for filename
Attempts to open file and read it to check if it exists
If file already exists, print error and close file
If file does not exist, opens and begins writing to file
Writes integers 1 to 100, one number per line, to file
Closes and exits file
```

Complexity:

For head.c, the time complexity is $\mathcal{O}(1)$ since it only reads 3 lines, independent of size.

For tail.c, the time complexity is $\mathcal{O}(n)$ since it depends on the number of lines of the file (i.e., $n \text{ num} = 344$).

For out.c, the time complexity is $\mathcal{O}(1)$ since it writes exactly 100 numbers every time.

2 Exercise 2:

Problem: The goal for this exercise is to dynamically allocate an array of floats that the user inputs. The makes sure the inputs are valid ($1 \leq n \leq 125$), avoids dividing by 0, and prevents it from being out of bounds.

Design:

```
Prompt user for array size
Checks to make sure input is between 1 and 125
If input is invalid, prints error message and terminates
Allocate n floats with malloc() command
Checks to make sure allocation is successful
In for loop, loops from i = 0 to n - 1
If i ==0, assigns i a value of 1 to avoid dividing by 0
If i != 0, computes  $1 + i^2 + (i^3)/3$  using pow
Prompt user for element number
Checks to make sure its between 1 and input n
Prints requested value using equation
Free allocated memory
Exit program
```

Complexity:

The time complexity for `bounds.c` is $\mathcal{O}(n)$ since the computational time increases linearly with respect to the size of the array, requiring n iterations.

3 Exercise 3:

Problem: The goal for this exercise is to create an $n \times n$ spiral matrix that has integers 1 to n^2 . The spiral is clockwise starting from the center of the matrix. The program takes an input through the terminal after using `./` to run the script. The program checks to make sure the input is an integer between 1 and 100, dynamically allocate to matrix, and store the information in a spiral format. The program writes the output to a file called `ans.out`.

Design:

```
Check to make sure only one argument is inputted to the command line
Checks to make sure it is a valid int and is between 1 and 100
If not in bounds, prints error message and terminates
Allocates n row pointers and n integers per row (2x2 matrix)
Finds center position of matrix
Initialize direction (d [N,E,S,W])
Initialize counter (c) and direction length (s)
Puts first value at center of matrix
Increase direction length every two turns
Keep repeating process until  $n^2$  elements are in matrix
Find number of digits in  $n^2$ 
Prints matrix to ans.out script and fixes width
Free allocated memory
```

Complexity:

The time complexity for `shell.c` is $\mathcal{O}(n^2)$ since the matrix contains n^2 elements and each element is written and printed once. This means that if n doubles, the runtime increases four times.